Введение в старницы

Чаще всего при решении проблем с управлением памятью ОС принимают одно из нескольких решений. Первый подход который мы рассмотрели в прошлый раз - разбиение пространства на кусочки динамичной длины, что приводит к сегментированности выделенного пространства. К сожалению такое решение имеет некоторые трудности. В частности при разделении пространства на кусочки разного размера мы приходим к фрагментации и как следствие выделение памяти становится все тяжелее со временем.

Таким образом следует рассматривать и другие способы управления адресным пространством - разделять его на кусочки фиксированного размера. Такое разделение памяти на кусочки называется еще разделением на страницы или **paging**. Вместо того чтобы разделять на куски разного размера память делится на куски фиксированного размера, которые называются страницами (page). Соответственно физическая память представляется нам как массив слотов фиксированного размера, которые называются **page frames**. Каждый такой слот может содержать страницу виртуальной памяти.

Суть проблемы:
Как виртуализировать память с использованием страниц? Как выполнять виртуализацию и при этом избегать сегментацию. Какие базовые техники использовать? Как заставить эти техники работать хорошо с минимальным использованием пространства и накладных расходов.

Простой пример

На изображении приведен пример адресного пространства размером 64 байта. Оно разделено на кусочки по 16 байт (Виртуальные страницы 0-3). Реальное адресное пространство естественно много больше, но для простоты мы возьмем такой размер.

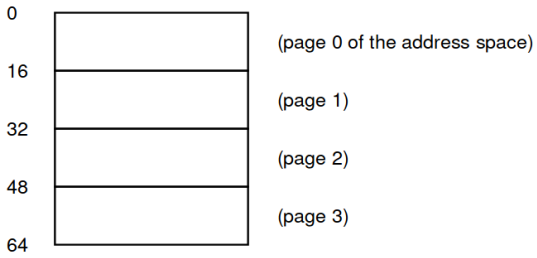
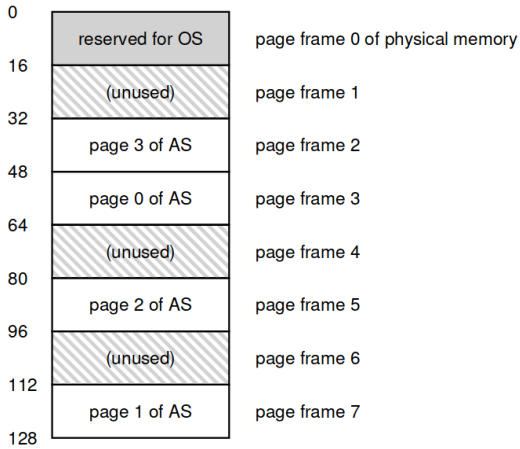


Figure 18.1: **A Simple 64-byte Address Space**



Физическая память же, как показано на следующем изображении также содержит несколько слотов фиксированного размера, в данном случае 8 page frame, которые в сумме составляют 128 байт. Как видно из изображения эти страницы располагаются в разных местах в физической памяти. Также на рисунке изображено некоторое количество памяти, занятой ОС.

Разбиение на страницы имеет несколько преимуществ по сравнению с предыдущим подходом. Во-первых это гибкость: при использовании страниц более эффективно поддерживается абстракция адресного пространства, независимо от того сколько процесс использует адресного пространства. Например нам не потребуется делать предположений о направлении роста кучи и стека и о характере их использования.

Во-вторых преимуществом является простота обслуживания адресного пространства, которое организует paging (постраничное разбиение). Например когда ОС желает поместить наше маленькое адресное пространство размером 64 байта в 8-страничную физическую память, она просто найдет 4 свободные страницы. Возможно даже ОС ведет список свободных страниц в т.н. free list и для нашей задачи просто возьмет первые 4 свободных страницы из этого списка. В данном примере ОС поместила virtual page 0 (VP) адресного пространства (AS) в физический кадр (PF) 3, vp1->pf7, vp2->pf5, vp3->pf2. Pf 1,4,6 остаются свободны.

Для того чтобы отслеживать куда каждая VP каждого адресного пространства помещена в физическую память, ОС хранит для каждого процесса некоторую структуру данных, которая называется **page table**. Основной целью этой таблицы - хранение соответствия адресов (**address translations**) для каждой виртуальной страницы в адресном пространстве. Таким образом мы всегда знаем где в физической памяти располагается каждая виртуальная страница.

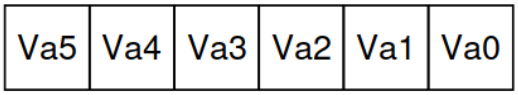
Важно помнить, что эта таблица уникальная для каждого процесса. Если какой-то другой процесс будет запущен помимо того что у нас уже есть в примере выше, то ОС будет обслуживать его уже с другой таблицей страниц, поскольку его виртуальные страницы будут очевидно указывать на другие физические страницы.

Рассмотрим пример. Возьмем некоторый процесс с маленьким адресным пространством (64 байта) и этот процесс осуществляет доступ к памяти:

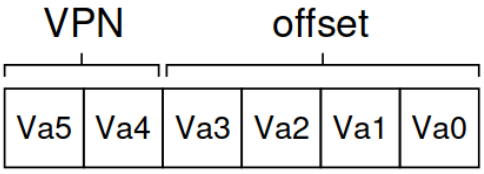
Movl <virtual address>, %eax

В частности давайте уделим внимание явной загрузке данных из адреса <virtual address> в регистр eax (и проигнорируем инструкции которые предшествовали этой).

Для трансляции этого виртуального адреса для начала нам надо разделить его на несколько компонентов: **virtual page number**(VPN) и сдвиг (**offset**) в пределах этой страницы. Для такого адресного пространства, размером 64 байта, нам нужно 6 бит для указания виртуального адреса (2^6). Таким образом наше адресное пространство представлено следующим образом:



В этом изображении virtual address 5 - старший бит адресного пространства, а va0 - младший. Поскольку мы знаем размер страницы (16 байт), мы можем далее разделить наше пространство следующим образом:

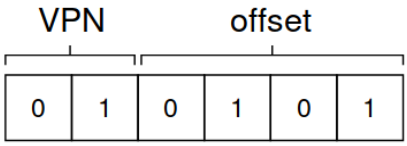


При размере страницы равном 16 байт в 64-байтном пространстве мы должны иметь возможность адресовать 4 страницы и 2 старших бита позволяют нам это сделать. Таким образом мы имеем 2 бита в качестве virtual page number. Оставшиеся биты говорят нам в каком конкретно байте мы заинтересованы, в данном случае эту роль выполняют оставшиеся 4 байта, которые называются сдвигом (offset).

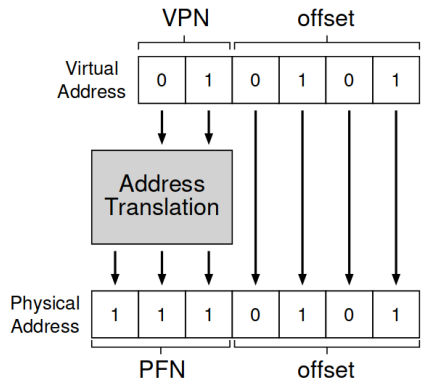
Когда процесс генерирует виртуальный адрес, ОС совместно с железом должны транслировать его в осмысленный физический адрес. Например представим, что в примере выше была выполнена загрузка виртуального адреса 21

movl 21, %eax

После приведения адреса 21 в бинарный вид мы получаем число 010101 и теперь можем рассмотреть как этот виртуальный адрес разбивается на virtual page number и сдвиг:



Таким образом наш виртуальный адрес 21 находится на 5-м байте (0101) виртуальной страницы 01 или 1. Теперь мы можем посмотреть page table и найти какой в каком physical frame располагается наша virtual page 1. В page table есть запись, что наш physical frame number (PFN) (или его еще называют physical page number PPN) равен 7 (vp1->pf7) а в бинарном виде это 111. Теперь мы можем транслировать virtual address, заменяя VPN числом PFN а затем выполнять загрузку в физическую память.



Обратите внимание на то что сдвиг остается точно таким же. Т.е. он не транслируется, поскольку сдвиг просто напросто говорит нам какой байт внутри страницы нам нужен. Наш финальный адрес получается 1110101 (117) и это и есть то самое место из которого мы хотим получить данные.

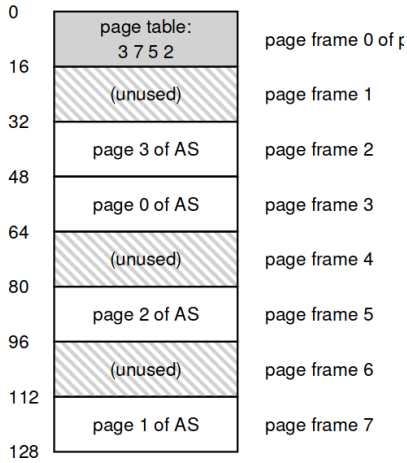
Теперь обладая этими знаниями мы можем ответить на несколько вопросов об paging. Например на вопрос о том где располагаются эти page tables. Каково типичное содержание этой таблицы? Делает ли использование этой технологии систему медленнее?

Где располагаются page tables

Размеры таких таблиц могут быть неприятно большими. Гораздо большими, чем таблицы которые используются при сегментированных подходах. Например представьте себе типичное 32-битное пространство с 4-Кб страницами. Это виртуальное адресное пространство делится на 20-битные VPN и 12-битные сдвиги.

20-битные VPN предполагают что будет возможно 2^20 трансляций, которые ОС должна обслуживать для каждого процесса, которых может быть большое количество. Предполагая, что нам потребуется 4 байта для записи **page table entry (PTE)**, чтобы хранить физическую трансляцию и возможно какую-то еще дополнительную информацию. В итоге получается что для каждой page table будет необходимо 4Mb памяти. Это довольно много. Представьте что у вас запущено 100 процессов, тогда эти 4 мб превратятся в 400, и только лишь для хранения трансляций. И это уже не говоря о том, что в 64-битном пространстве это число вырастет куда больше.

Из-за того что page tables настолько большие, мы не сможем их хранить на MMU чипе. Вместо этого мы будем хранить таблицу где-нибудь в памяти. Предположим что page table располагается где-то в физической памяти, которой управляет ОС.



Что на самом деле в page table

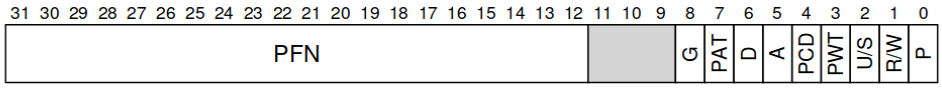
Теперь давайте обсудим о самой организацию таблицы. Эта таблица - всего лишь структура данных, которые используются для отображения виртуального адреса (или на самом деле virtual page number) в физический адрес (physical frame number). Таким образом любая структура может подойти. Самой простейшей формой называется линейная таблица страниц **(linear page table)**, которая по сути является массивом. ОС индексирует массив по VPN и смотрит на запись page-table entry (PTE), которая соответствует этому индексу. Это делается для того чтобы найти желаемый номер кадра (physical frame number) (PFN). Мы будем рассматривать в данном случае только линейную структуру для простоты.

Что касается содержимого каждого PTE, там есть еще несколько различных бит, которые следует понимать на некотором уровне. **Valid bit** типичен для индикации того что трансляция действительна. Например когда программа запускается, у нее будет код и стек с одной стороны и кучу с другой стороны адресного пространства. Все остальное пространство между ними будет помечено как недействительное. И если процесс попробует к нему обратиться, сработает перехват (trap) в ОС и скорее всего такой процесс будет завершен. Таким образом valid bit является ключевым в поддержке разреженности адресного пространства. Просто-напросто помечая все неиспользуемые страницы как недействительные, мы снимаем с себя необходимость выделять физические кадры (physical frames) для этих страниц и таким образом мы экономим очень много памяти.

Также существуют биты защиты **(protection bits)**, которые указывают может ли быть страница прочитана, записана или исполнена. И аналогично, в зависимости от бита если выполнить не предусмотренное действие, то через перехвата ОС снова убьет этот процесс.

Present bit указывает на то где находится страничка - в физической памяти или на диске. (например она может быть выгружена в swap). Swap позволяет ОС выгружать освобождать физическую память перемещая редко используемые страницы на диск. Еще

существует **dirty bit** который указывает на то, была ли изменена с тех пор как эта страница была загружена в память.
Reference bit (successful bit) иногда используется для отслеживания обращений к странице и помогает определить какие страницы наиболее популярны и должны оставаться в памяти. Это знание важно вовремя замены страниц.



На этом рисунке изображена запись в таблице страниц типичная для архитектуры x86. Она содержит present bit, read/write bit, user/supervisor bit (U/S) который показывает, могут ли программы в режиме пользователя получать доступ к этой странице. Accessed bit (A), Dirty bit (D), PFN и некоторые другие биты, которые определяют как будут работать аппаратные техники кэширования.

Paging: Все еще медленные

Кроме того что page table может занимать много места в памяти, они могут еще и быть довольно медленными. Например снова рассмотрим инструкцию:
Movl 21, %eax

В этом примере для считывания желаемых данных из памяти система должна сначала транслировать виртуальный адрес 21 в корректный физический адрес (117). Т.е. перед тем как считать данные с адреса 117 система должна считать считать PTE (page table entry) из таблицы страниц процесса, выполнить трансляцию, а затем загрузить данные из физической памяти.

Для того чтобы это сделать, оборудование должно знать где находится таблица страниц для в настоящее время работающего процесса. Предположим что базовый регистр таблицы страниц содержит физический адрес начала таблицы страниц. Для того чтобы найти расположение желаемого PTE оборудование выполнит следующие инструкции:

```
VPN     = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

В нашем примере VPN_MASK = 0x30, (110000b), что позволит нам извлечь биты VPN из виртуального адреса. SHIFT принимает значение 4 (число бит сдвига), таким образом чтобы сдвинуть результат произведения и получения итогового значения VPN.
Например для виртуального адреса 21 (010101) применение маски преобразует число в 010000, а затем сдвиг преобразует в 01 или VP1, как мы того и хотели. Затем мы используем это число в качестве индекса в массиве PTE’s, на которые указывает базовый регистр таблицы страниц.

После того как мы узнали физический адрес, оборудование теперь может считать PTE из памяти, извлечь PFN, и соединить его со сдвигом (взятым из виртуального адреса)

чтобы наконец сформировать желаемый физический адрес.

```
offset   = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

В частности мы можем представить что PFN сдвигается влево на SHIFT, а затем умножается побитовым или на сдвиг.

Наконец, оборудование может извлечь желаемые данные из памяти и поместить их в регистр eax. Программа в этот момент успешно получила значение, загруженное из памяти.

```
1  // Extract the VPN from the virtual address
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // Form the address of the page-table entry (PTE)
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // Fetch the PTE
8  PTE = AccessMemory(PTEAddr)
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access is OK: form physical address and fetch it
17     offset   = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

Подводя итог, мы описали базовый пример того что происходит на каждое обращение к памяти. На каждое обращение к памяти (будь то загрузка инструкции или явная загрузки из памяти или в память) механизм paging требует от нас выполнять одно дополнительное обращение к памяти для того чтобы извлечь информацию о трансляции из таблицы страниц. Это на самом деле является огромной дополнительной нагрузкой и таким образом мы замедляем работу в 2 и более раз.

Таким образом существует 2 реальные проблемы, которые нужно решить. Без продуманной архитектуры оборудования и программного обеспечения таблицы страниц приведут к тому что система будет работать крайне медленно и использовать чрезмерно много памяти. Хотя, казалось бы, это отличное решение для нашей задачи виртуализации памяти, эти две ключевые проблемы должны быть решены.

Трассировка памяти

Перед завершением этой лекции рассмотрим простой пример обращения к памяти для демонстрации всех обращений к памяти, возникающие при использовании технологии

paging. Рассмотрим набросок кода на C:

```
int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;
```

Попробуем дизассемблировать программу и посмотреть на инструкции, которые составляют цикл инициализации массива.

```
1024 movl $0x0,(%edi,%eax,4)
1028 incl %eax
1032 cmpl $0x03e8,%eax
1036 jne  0x1024
```

Первая инструкция помещает значение =0 ($0x0) в виртуальный адрес в памяти, где располагается этот массив. Этот адрес вычисляется произведением содержимого регистра eax на 4 с последующим сложением с регистром edi. Edi содержит базовый адрес массива, а eax содержит индекс (i). Почему умножили на 4?

Следующая инструкция увеличивает на единицу индекс массива, содержащийся в eax. Третья инструкция сравнивает содержимое eax с hex-представлением числа 1000. Если сравнение показывает, что эти числа не равны, то происходит переход к началу цикла.

В рамках данного примера представим что у нас есть адресное пространство размера 64kb. Размер страницы = 1kb.

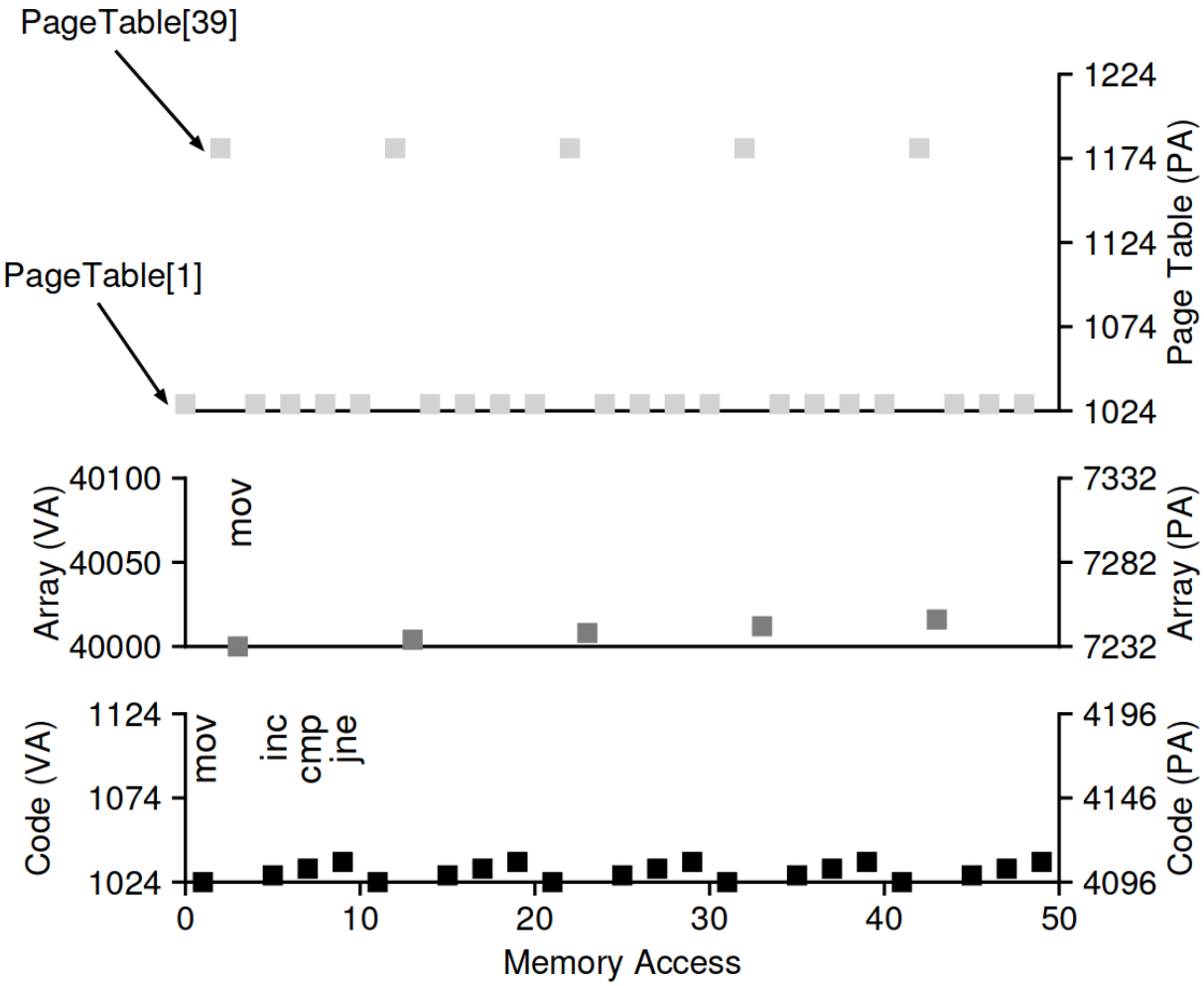
Все что нам нужно теперь знать - содержимое таблицы страниц и ее расположение в физической памяти. Предположим у нас есть линейная (основанная на массиве) таблица и что она располагается в физической памяти по адресу 1кб (1024).

Что касается содержимого таблицы, то нас волнует всего несколько страниц в рамках данного примера. Во-первых страница с кодом. Поскольку размер страницы 1Кб, то виртуальный адрес 1024 располагается на второй странице виртуального адресного пространства. (VPN=1, т.к. VPN=0 - первая страница). Предположим, что virtual page отображается в physical frame 4 (vpn1->pfn4).

Далее рассмотрим сам массив. Его размер 4000 байт (1000 int), и мы предполагаем что он располагается по виртуальному адресу с 40.000 до 44.000 (не включая последний байт). VPN=39..42. Предположим что VPN39->PFN7, VPN40->PFN8, VPN41->PFN9, VPN42->PFN10.

Теперь мы готовы отслеживать запросы в памяти программы. Когда она запущена, каждый запрос инструкции будет генерировать 2 обращения к памяти: первый к таблице страниц для того чтобы найти физический кадр в котором располагается нужная нам инструкция. Второе обращение к памяти непосредственно получит эту самую инструкцию для последующего выполнения на CPU. В дополнение, есть одно явное обращение к памяти в виде инструкции mov. Эта инструкция включает в себя еще одно обращение к

другой таблице страниц (для того чтобы транслировать массив виртуальных адресов в корректные физические) а потом получить доступ уже к самому массиву.



На изображении показаны обращения для первых 5 циклов. Нижний график показывает обращения к инструкциям в памяти - виртуальный адрес слева, физический - справа. Центральный график показывает обращение к массиву - виртуальный адрес слева, физический - справа. Верхний график показывает доступ к самой таблице страниц в памяти - только физические адреса, т.к. Виртуальных нет.

Ось x показывает обращения к памяти. Всего нарисовано 5\10.

Один цикл включает в себя 10 обращений к памяти, включая 4 получения инструкции, одно явное обновление памяти и 5 обращений в таблице страниц для трансляции 4 инструкций и обновления страницы.

Итоги

Мы представили концепцию paging, как одно из решений виртуализации памяти. Такой подход имеет множество преимуществ по сравнению с предыдущим методом - как минимум это отсутствие внешней фрагментации, поскольку по задуманной архитектуре уже этого не позволяют. Во-вторых это достаточно гибкое решение которое позволяет реже использовать адресные пространства.